# Lecture 05 - Interfaces and Generics

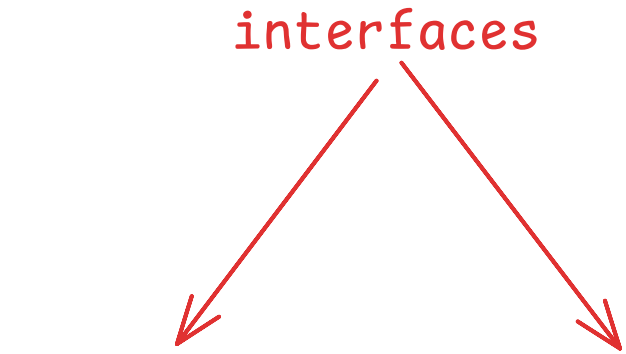
1

# Interfaces

```
type Dog struct{}

type Cat struct{}

func (Dog) Speak() string {
    return "wof"
}

func (Dog) Walk() {
    ...
}

func (Cat) Speak() string {
    return "meow"
}
```

```
type Speaker interface{
    Speak() string
}

func MakeSound(s Speaker) {
    fmt.Println(s.Speak())
}
```

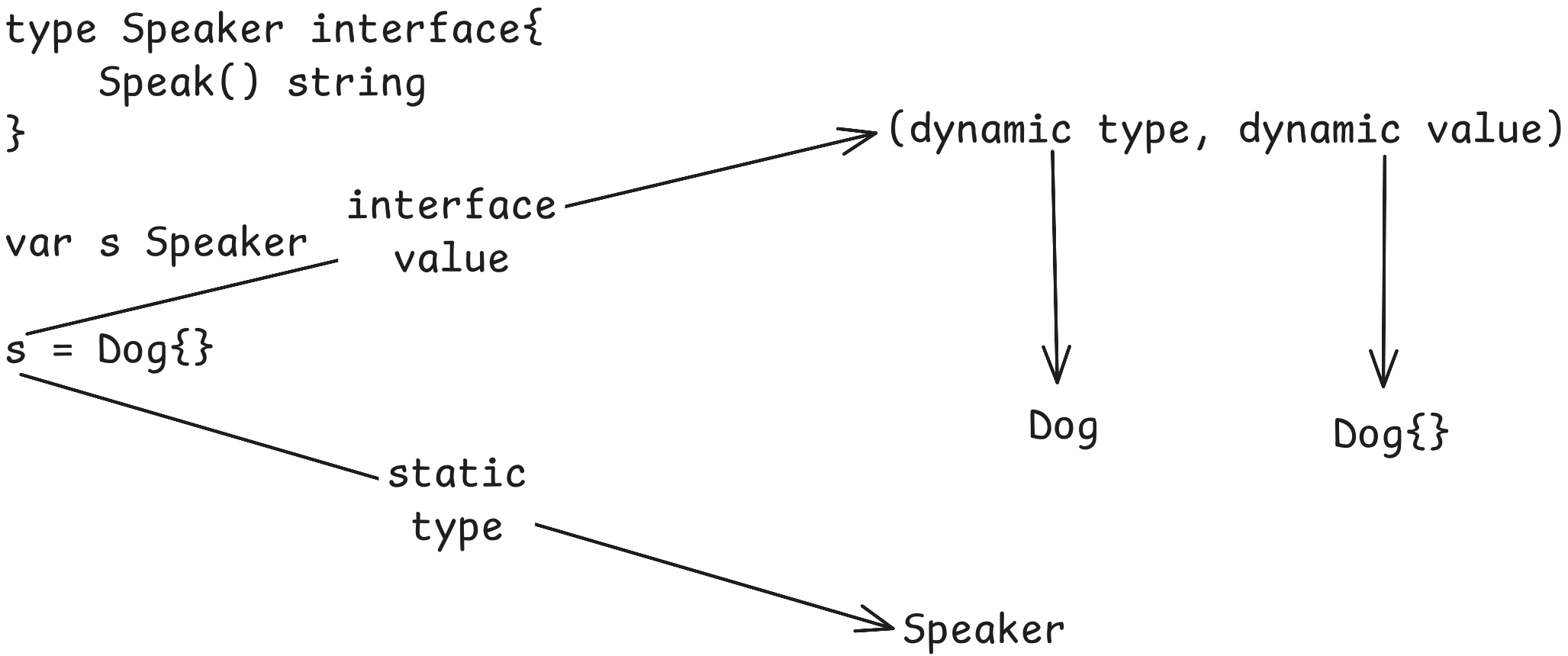
If a type has methods required by the interface then implements it




implicit

explicit

```
class Dog implements Speaker
```


type Speaker interface{
Speak() string
}
var s Speaker
s = Dog{}
interface value
(dynamic type, dynamic value)
Dog
Dog{}
static type
Speaker

interface{}

alias

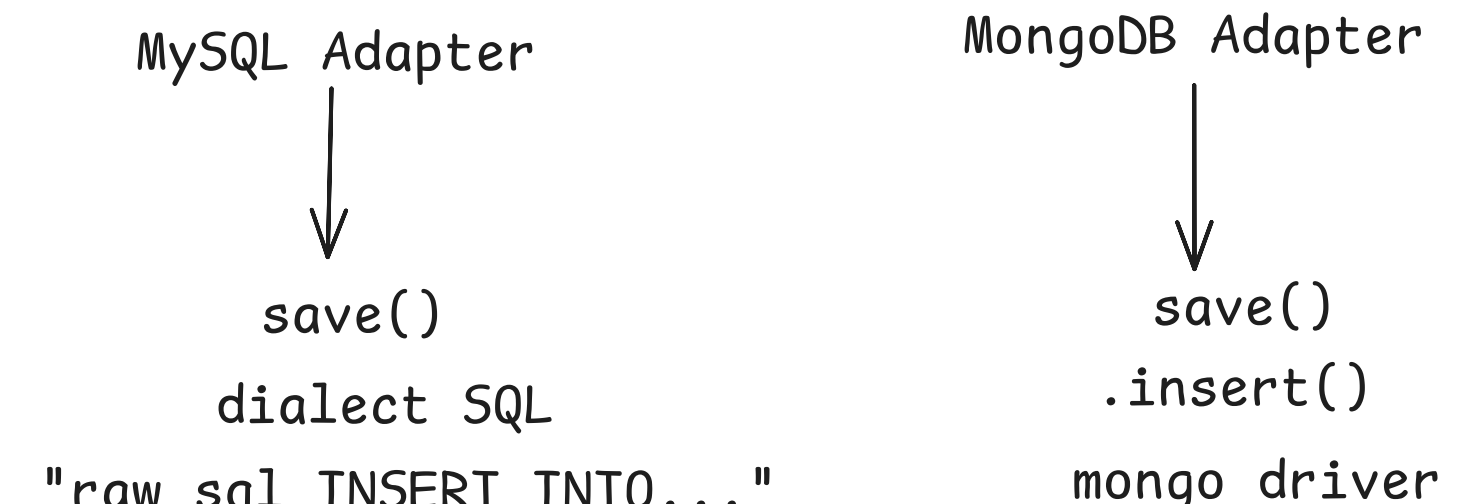
```
func someFunction(x any) {

}
```

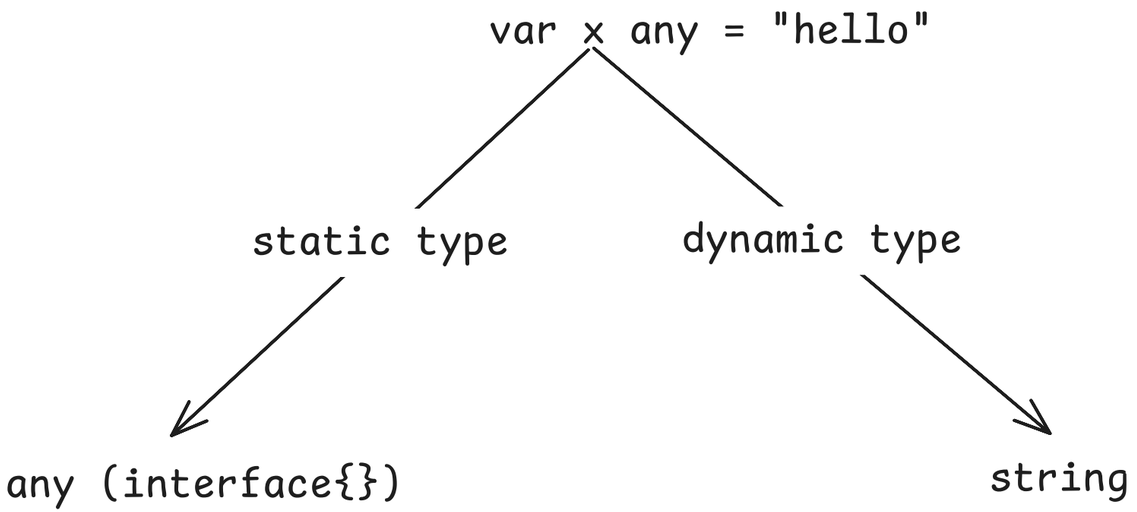
<same>

```
func someFunction(x interface{}) {

}

type stringV2 string

func (s string) len() {
    return 0
}

var str stringV2

str := "hello!"

str.len() // 0
```

```
class Controller

if <...> {
    // write to mongoDB
} else {
    // write to mysql
}
```

```
class Store

func store(s AdapterInterface) {
    <...preprocessing>
    s.save()
}

func storeToMySQL()

func storeToMongoDB()
```


MySQL Adapter
save()
dialect SQL
"raw sql INSERT INTO..."
MongoDB Adapter
save()
.insert()
mongo driver

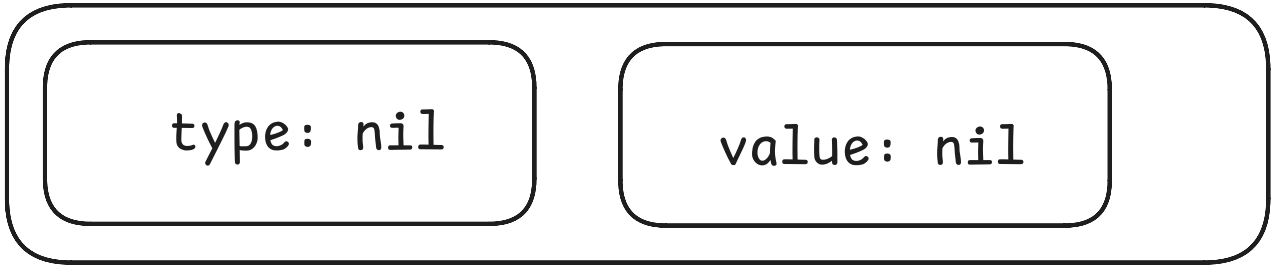



type assertion

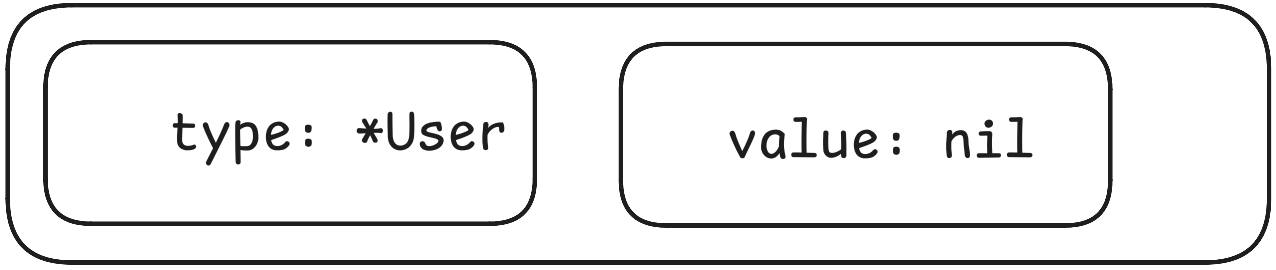
safe-type assertion:
comma-ok idiom

```
var x any = 42

s := x.(string)

fmt.Println(s) // hello

safeWay, ok := x.(string)

if ok {
    fmt.Println(safeWay)
}
```

```
var x any

x == nil // true
```




```
var p *User = nil

var x any = p

x == nil // false
```



## Type Switches

different behavior depending on concrete type

```
if s, ok := v.(string); ok {
    // some behavior for string
}

if s, ok := v.(int64); ok {
    //  some behavior for int64
}
```

```
switch s := v.(type) {
case string:
    ...
case int, int32, int64:
    ...

<some other cases...>

default:
    ...
}
```

```
type Reader() interface {
     Read()
}

type Writer() interface {
    Write()
}
```

```
var r Reader

c, ok := r.(Writer)

if ok {
    r.Write()
}
```